Algorithms 2

Reductions

Daniel Rosenberg & Michael Trushkin

Ariel University

1. NP-completeness

1.1. NP-completeness

- As we saw, most problems are *hard*!

1.1. NP-completeness

- As we saw, most problems are *hard*!

Definition 1.1.1 (NP class) A language L is said to be in $\mathbf{NP}$ if we have a polynomial-time algorithm M such that

$$x \in L \Leftrightarrow \exists y \; s.t \; |y| < p(|x|) \text{ and } M(x, y) = 1$$

where p is some polynomial

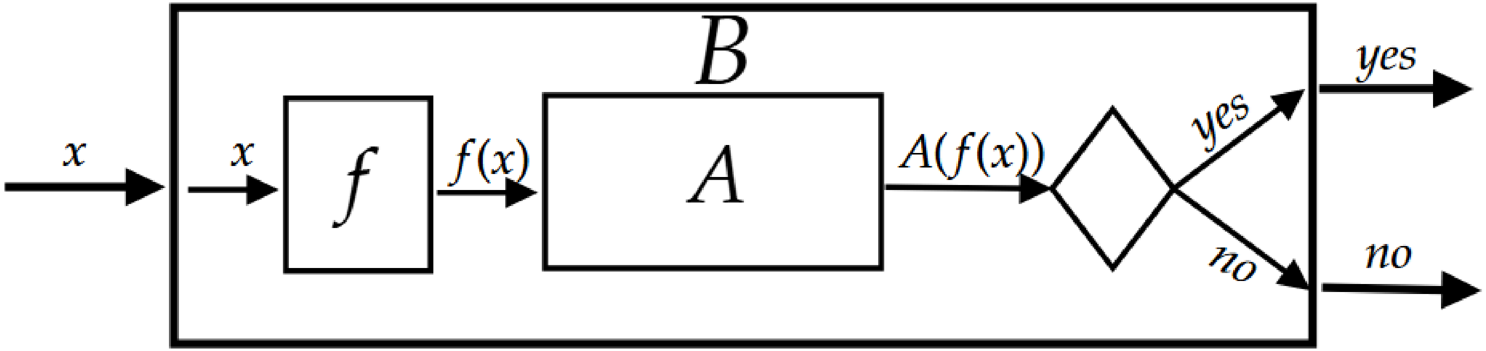
- In most literture y is called a *witness* and M is called *veryfing algorithm*, where y plays the role of the answer, and M should just verify if the answer is correct.
- We talked about a few:
  - k-clique
  - CNF-SAT
  - k-CNF-SAT

2. Reductions

2.1. Reductions

- We saw we can “translate” one languge into another with reductions

Definition 2.1.1 (polynomial time reduction) Given two languages $L_1, L_2 \in \mathbf{NP}$, we write $L_1 \leq_p L_2$ if there exists a function $f : \{0,1\}^* \to \{0,1\}^*$ and a polynomial $p : \mathbb{N} \to \mathbb{N}$, such that:
- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every $x \in \{0,1\}^*$, f runs in $p(|x|)$ time.



,

2.2. Hard Langauges

- For any complexity class, there are hard languages

Definition 2.2.1 (NP-hard) A language $L \subseteq \{0,1\}^*$ is said to be NP-hard if $L' \leq_p L$ for every $L' \in \mathbf{NP}$

Definition 2.2.2 (NP-complete) A language $L \subseteq \{0,1\}^*$ is said to be NP-complete if $L \in \mathbf{NP}$ and L is NP-hard

- We saw that CNF-SAT and k-CNF-SAT are NP Complete

2.3. Independent set

- For a graph G, let $\alpha(G)$ denote the size of the maximum independent set in G.

Definition 2.3.1 IS $:= \{< G, k >: \alpha(G) \geq k\}$.

Theorem 2.3.2 IS is in $\mathbf{NPC}$.

2.4. Independent set

- The proof that IS is in **NP** is left as homework.
- In order to show that IS hard we need to show that

$$L \leq_p \mathrm{IS}$$

for some **NP** hard languge.